Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 08

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Identificar la funcionalidad de Django REST Framework (serializers, vistas, ViewSets y routers) para exponer los datos de una aplicación empresarial como servicios web.**
2. **Implementar serializers y endpoints CRUD (vistas genéricas y ViewSets con routers) sobre modelos ya existentes, con validación y relaciones anidadas.**
3. **Aplicar búsqueda, ordenamiento y filtros por parámetro, y verificar los códigos de estado HTTP con un cliente de pruebas.**

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la Semana 8 (Django REST Framework: API y REST, métodos HTTP y códigos de estado, serializers, vistas genéricas, ViewSets, routers y filtros) antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, Django REST Framework (pip install djangorestframework), un editor de código (Visual Studio Code), un cliente de pruebas de API (Thunder Client de VS Code, Postman o curl) y una cuenta de GitHub para el control de versiones.
- La aplicación desarrollada desde la Semana 3 (CRUD), ampliada con relaciones en la Semana 4, registrada en el Django Admin en la Semana 5, con Templates que heredan de base.html desde la Semana 6 y con el campo de estado y el QuerySet personalizado de la Semana 7, y, cuando corresponda, la investigación propia de siete entidades.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código (serializers.py, api_views.py y las URLs) y de la respuesta de la API en el navegador o en el cliente de pruebas, mostrando el método HTTP y el código de estado obtenido.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

VII. Procedimiento

Desde la Semana 3, cada estudiante ya cuenta con una aplicación con CRUD, relaciones, Django Admin, Templates con herencia y, desde la Semana 7, consultas avanzadas y un QuerySet personalizado. Hasta ahora esa aplicación entrega páginas HTML. En esta sesión no se agregan entidades nuevas: se exponen las que ya existen como una API REST que entrega datos en formato JSON, para que otros programas (una app móvil, o React en la Semana 9) puedan consumirlas.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante retoma la aplicación de las Semanas 4 a 7 y le agrega una API con Django REST Framework: serializers, vistas genéricas, ViewSets, routers, filtros y una acción propia. En la segunda parte, deberá hacer lo mismo sobre su investigación propia (las siete entidades registradas en el Admin en la Semana 5), cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que la API no reemplaza al ORM ni a las consultas que ya construyó: las reutiliza. El mismo Model, las mismas relaciones y el mismo QuerySet personalizado de la Semana 7 son los que alimentan a los serializers y a los ViewSets. En clase la teoría se demuestra con una aplicación de cafetería; cada estudiante la traslada a su propia aplicación con ayuda de la siguiente tabla de equivalencias.

Concepto trabajado en clase	Equivalente en su aplicación
Recurso simple con una regla de negocio (en clase: Producto, con precio mayor que cero)	Una entidad de su aplicación con campos simples y una regla que pueda validarse (precio, cantidad, capacidad, título).
Recurso con relaciones, de solo lectura (en clase: Pedido con sus detalles)	Su entidad principal junto con su relación 1:N, 1:1 o su modelo intermedio N:M.
Campo de estado para filtrar con ?estado= (en clase: estado de Pedido)	El campo de estado que trabajó en la Semana 7.
Método del QuerySet personalizado (en clase: pagados())	Un método del QuerySet personalizado de la Semana 7.

Cliente (navegador o Thunder Client)

→ URL de la API (ej. /api/productos/)

→ Router → ViewSet

→ Serializer (valida y convierte a JSON)

→ Manager / QuerySet personalizado

→ Django ORM

→ SQLite

→ Response (JSON y código de estado)

Al finalizar el laboratorio, el estudiante deberá ser capaz de crear serializers con validación, exponer un recurso con vistas genéricas y con ViewSets y routers, probar el CRUD verificando los códigos de estado HTTP, optimizar una API con relaciones anidadas y agregar búsqueda, ordenamiento, filtros por parámetro y una acción propia.

Enunciados

PARTE 1 — Una API REST sobre la aplicación de las Semanas 4 a 7

Ejercicio 1 — Instalar Django REST Framework e identificar los recursos

Abra la aplicación desarrollada hasta la Semana 7 e identifique dos recursos para exponer: un recurso simple (con la ayuda de la tabla de equivalencias) y otro con relaciones. Verifique que cuenta con el campo de estado y el QuerySet personalizado de la Semana 7; si no los tiene, créelos ahora. Instale el paquete con pip install djangorestframework, agregue 'rest_framework' a INSTALLED_APPS y actualice requirements.txt. Documente con capturas los dos recursos elegidos y el archivo settings.py.

Ejercicio 2 — Crear un ModelSerializer y serializar en el shell

Cree en serializers.py un ModelSerializer para el recurso simple. En el shell de Django (python manage.py shell), serialice un objeto y luego un QuerySet completo con many=True. Documente con capturas ambas salidas y explique con sus palabras por qué se necesita many=True en el segundo caso.

Ejercicio 3 — Deserializar y validar

Agregue al serializer una validación propia con un método validate_<campo> que exprese una regla de negocio (por ejemplo, precio mayor que cero o cantidad de al menos uno). En el shell, cree el serializer con datos (data=...) y pruebe un caso válido y uno inválido. Muestre el resultado de is_valid(), validated_data y errors, y guarde el caso válido con save().

Ejercicio 4 — Exponer el recurso con vistas genéricas

Cree una vista con ListCreateAPIView y otra con RetrieveUpdateDestroyAPIView para el recurso simple, agregue sus URLs bajo /api/ y pruebe en el navegador la lista y el detalle (API navegable de DRF). Documente las URLs, el código y las capturas del navegador.

Ejercicio 5 — Reemplazar las vistas por un ViewSet y un router

Reemplace las dos vistas genéricas por un ModelViewSet registrado en un DefaultRouter. Abra /api/ y muestre los recursos registrados. Explique con sus palabras qué rutas y qué métodos HTTP se obtienen sin haberlos escrito a mano.

Ejercicio 6 — Probar el CRUD y los códigos de estado

Con Thunder Client, Postman o curl, ejecute sobre el recurso simple: GET de la lista, GET de un detalle, POST válido, POST con datos inválidos, PUT o PATCH, DELETE y GET de un id inexistente. Complete una tabla con la petición, el código de estado obtenido y una captura de cada respuesta. Explique con sus palabras qué diferencia hay entre esta respuesta y la de un Template, y por qué esta API queda abierta hasta que se trabaje la seguridad.

Ejercicio 7 — Un recurso de solo lectura con relaciones anidadas

Cree un serializer para el recurso con relaciones que incluya datos de la entidad relacionada de forma anidada (por ejemplo, un pedido con sus detalles) y exponga el recurso con un ReadOnlyModelViewSet. Escriba get_queryset() con select_related() o prefetch_related() según corresponda. Mida cuántas consultas ejecuta la serialización de todos los registros con Pedido.objects.all() (o su equivalente) y con el queryset optimizado, usando connection.queries, y documente el antes y el después.

Ejercicio 8 — Búsqueda, ordenamiento, filtro por parámetro y una acción propia

En el ViewSet del recurso simple agregue SearchFilter y OrderingFilter (search_fields y ordering_fields). En el ViewSet del recurso con relaciones, agregue en get_queryset() un filtro por el parámetro ?estado= (o el campo de estado de su aplicación) y una acción con @action(detail=False) que reutilice un método del QuerySet personalizado de la Semana 7. Documente al menos tres URLs con sus resultados, incluyendo una que combine parámetros con &.

PARTE 2 — Una API REST sobre la investigación propia (registrada en el Admin en la Semana 5)

A partir de aquí trabajará sobre la base de datos de su investigación propia: las cinco entidades originales, la entidad de la relación OneToOneField y el modelo intermedio de la relación ManyToManyField. El objetivo es aplicar las mismas técnicas sobre ese modelo de datos, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

Ejercicio 9 — Identificar los recursos de la investigación propia

Complete la tabla de equivalencias con su investigación: indique qué entidad expondrá como recurso simple con una regla de validación, qué entidad expondrá como recurso de solo lectura con relaciones, qué campo de estado usará para filtrar y qué método del QuerySet personalizado reutilizará en la acción. Si algún campo o método no existe, agréguelo antes de continuar.

Checklist antes de continuar — Parte 2

Criterio	Verificación
1. Django REST Framework está instalado, 'rest_framework' está en INSTALLED_APPS y requirements.txt está actualizado.	Captura de settings.py y de requirements.txt.
2. Al menos tres recursos de tu investigación tienen un ModelSerializer.	Código de serializers.py.
3. Al menos un serializer tiene una validación propia (validate_<campo>), probada con un dato válido y uno inválido.	Capturas de is_valid() y errors.
4. Al menos dos recursos se exponen con ModelViewSet y DefaultRouter bajo /api/.	Código de api_views.py y captura de /api/.

5. Se verificaron con un cliente de pruebas los códigos 200, 201, 400, 404 y 204.	Tabla de peticiones y códigos con capturas.
6. Existe un recurso de solo lectura con relación anidada y get_queryset() optimizado; se midió el número de consultas antes y después.	Capturas de len(connection.queries).
7. Al menos un ViewSet tiene SearchFilter y OrderingFilter funcionando.	Capturas de las URLs con ?search= y ?ordering=.
8. Hay un filtro por parámetro (?campo=) y una @action que reutiliza un método del QuerySet personalizado de la Semana 7.	Código + captura de la respuesta.
9. Se explica por escrito qué resuelve la API frente a los Templates y al Admin, y qué le falta todavía.	Párrafo de justificación.

Ejercicio 10 — Serializers y validación en la investigación propia

Cree ModelSerializers para al menos tres recursos de su investigación. En al menos uno agregue una validación propia con validate_<campo> y pruébela en el shell con un dato válido y uno inválido, mostrando is_valid() y errors.

Ejercicio 11 — ViewSets, router y códigos de estado en la investigación propia

Exponga al menos dos recursos con ModelViewSet y un DefaultRouter bajo /api/. Pruebe con un cliente de pruebas los códigos 200, 201, 400, 404 y 204 y documéntelos en una tabla con capturas.

Ejercicio 12 — Recurso de solo lectura con relaciones anidadas

Cree un recurso de solo lectura de su investigación que muestre datos de una entidad relacionada de forma anidada, con get_queryset() optimizado. Mida el número de consultas antes y después con connection.queries y documente ambos valores.

Ejercicio 13 — Filtros y acción propia en la investigación propia

Agregue SearchFilter y OrderingFilter a un ViewSet, un filtro por parámetro (?campo=) en get_queryset() y una acción con @action(detail=False) que reutilice un método del QuerySet personalizado de la Semana 7. Documente al menos tres URLs con sus resultados.

Ejercicio 14 — Publicar en GitHub

Actualice requirements.txt y README.md describiendo qué recursos se expusieron como API, qué serializers y ViewSets se implementaron y qué filtros y acciones se agregaron. Haga commit y push al repositorio. Incluya en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente serializers.py, api_views.py, las respuestas de la API con su código de estado y el número de consultas antes y después de optimizar).
- Repositorio en GitHub actualizado, con requirements.txt, README.md y el código correspondiente.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Conclusiones:

Desarrollo del laboratorio 8

Django REST Framework aplicado a la Clínica Veterinaria

En esta entrega expongo la aplicación de las semanas anteriores como una API REST en JSON. Mantengo los modelos y relaciones existentes, agrego validación y reutilizo el QuerySet personalizado de la Semana 7. El proyecto y las pruebas automatizadas están publicados en el repositorio indicado al final.

Parte 1 Una API REST sobre la aplicación de las semanas 4 a 7

Ejercicio 1 Instalar DRF e identificar recursos

GitHub main / Semana_08/core/settings.py | extracto de código

```
    'rest_framework',
    'gestion_clinica',  # <- Aquí debe estar la app de la Semana 03
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
```

GitHub main / Semana_08/requirements.txt | extracto de código

```
# Actualizado para Semana 08 - API REST con Django REST Framework
asgiref==3.12.1
contourpy==1.3.3
cycler==0.12.1
Django==5.2.17
fonttools==4.62.1
kiwisolver==1.5.0
matplotlib==3.10.8
networkx==3.6.1
numpy==2.4.3
packaging==26.0
pillow==12.1.1
pyparsing==3.3.2
python-dateutil==2.9.0.post0
six==1.17.0
sqlparse==0.6.0
tzdata==2026.3
djangorestframework==3.16.1
```

Agregué Django REST Framework a INSTALLED_APPS y djangorestframework==3.16.1 a requirements.txt. Elegí Paciente como recurso simple y PacienteServicio como recurso relacionado. El estado que filtro es PacienteServicio.estado; el QuerySet personalizado de la Semana 7 aporta solo_perros() y pesados().

```
pip install -r requirements.txt
# settings.py: INSTALLED_APPS incluye "rest_framework"
```

Ejercicio 2 ModelSerializer y serialización

GitHub main / Semana_08/gestion_clinica/serializers.py | extracto de código

```
class PacienteSerializer(serializers.ModelSerializer):
    """Serializer del recurso simple con validación de regla de negocio."""

    class Meta:
        model = Paciente
        fields = ["id", "propietario", "nombre", "especie", "raza", "edad", "peso"]

    def validate_peso(self, value):
        if value <= 0:
            raise serializers.ValidationError("El peso debe ser mayor que cero.")
```

PacienteSerializer hereda de ModelSerializer y declara los campos del modelo Paciente. Para un objeto se serializa una instancia; para un QuerySet se usa many=True porque contiene varias instancias.

```
from gestion_clinica.models import Paciente
from gestion_clinica.serializers import PacienteSerializer
paciente = Paciente.objects.first()
PacienteSerializer(paciente).data
PacienteSerializer(Paciente.objects.all(), many=True).data
```

Ejercicio 3 Deserializar y validar

GitHub main / Semana_08/gestion_clinica/serializers.py | extracto de código

```
10
11      class Meta:
12          model = Paciente
13          fields = ["id", "propietario", "nombre", "especie", "raza", "edad", "peso"]
14
15      def validate_peso(self, value):
16          if value <= 0:
```

La regla de negocio exige que el peso sea mayor que cero. is_valid() ejecuta las reglas del serializer. Los datos aceptados quedan en validated_data; los errores se consultan en errors. Después de validar, save() crea el registro.

```
datos = {"propietario": 1, "nombre": "Luna", "especie": "Gato", "raza": "Criollo", "edad": 2,
"peso": "8.50"}
s = PacienteSerializer(data=datos)
s.is_valid()  # True si existe el propietario indicado
s.validated_data
s.save()

invalidos = {**datos, "peso": "0"}
s_mal = PacienteSerializer(data=invalidos)
s_mal.is_valid()  # False
s_mal.errors  # error en el campo peso
```

Ejercicio 4 Exponer el recurso con vistas genéricas

GitHub main / Semana_08/gestion_clinica/api_views.py | extracto de código

```
16  # Vistas genéricas del ejercicio 4, conservadas bajo /api/genericos/ como demostración.
17  class PacienteListCreateAPIView(generics.ListCreateAPIView):
18      queryset = Paciente.objects.select_related("propietario").all()
19      serializer_class = PacienteSerializer
20
21
22  class PacienteDetailAPIView(generics.RetrieveUpdateDestroyAPIView):
23      queryset = Paciente.objects.select_related("propietario").all()
24      serializer_class = PacienteSerializer
```

La vista de colección usa ListCreateAPIView (GET y POST) y la de detalle usa RetrieveUpdateDestroyAPIView (GET, PUT/PATCH y DELETE). Para poder documentar esta parte, están disponibles en /api/genericos/pacientes/ y /api/genericos/pacientes/<id>/. El router del ejercicio 5 ofrece los endpoints principales en /api/pacientes/.

Ejercicio 5 ViewSet y DefaultRouter

GitHub main / Semana_08/gestion_clinica/api_urls.py | extracto de código

```
7       ServicioViewSet, VeterinarioViewSet,
8   )
9
10  router = DefaultRouter()
11  router.register("pacientes", PacienteViewSet, basename="paciente")
12  router.register("propietarios", PropietarioViewSet, basename="propietario")
13  router.register("medicamentos", MedicamentoViewSet, basename="medicamento")
14  router.register("veterinarios", VeterinarioViewSet, basename="veterinario")
15  router.register("historiales", HistorialMedicoViewSet, basename="historial")
16  router.register("servicios", ServicioViewSet, basename="servicio")
17  router.register("citas", CitaViewSet, basename="cita")
```

PacienteViewSet hereda de ModelViewSet. DefaultRouter genera automáticamente las rutas de colección y detalle y conecta los métodos HTTP a las acciones CRUD. El índice navegable queda en /api/.

```
GET /api/pacientes/       lista
POST /api/pacientes/      crea
GET /api/pacientes/1/     detalle
PUT o PATCH /api/pacientes/1/  actualiza
DELETE /api/pacientes/1/  elimina
```

Ejercicio 6 CRUD y códigos HTTP

GitHub main / Semana_08/gestion_clinica/tests.py | extracto de código

```
            paciente=cls.paciente, servicio=cls.servicio,
            cantidad=1, monto="45.00", estado="PAGADO",
        )

    def test_crud_and_http_status_codes(self):
        response = self.client.get("/api/pacientes/")
        self.assertEqual(response.status_code, 200)

        valid = {
            "propietario": self.propietario.pk,
            "nombre": "Luna", "especie": "Gato", "raza": "Criollo",
            "edad": 2, "peso": "4.25",
        }
        created = self.client.post("/api/pacientes/", valid, format="json")
        self.assertEqual(created.status_code, 201)
        patient_id = created.data["id"]

        invalid = {**valid, "nombre": "Nube", "peso": "0"}
        rejected = self.client.post("/api/pacientes/", invalid, format="json")
        self.assertEqual(rejected.status_code, 400)
        self.assertIn("peso", rejected.data)

        detail = self.client.get(f"/api/pacientes/{patient_id}/")
        self.assertEqual(detail.status_code, 200)
```

La suite APIEndpointTests contiene aserciones para los siguientes estados. El código de estas pruebas está en el repositorio; no se ejecutó en este entorno porque Django y Django REST Framework no están instalados.

Petición	Estado esperado en la prueba	Qué verifica
GET lista	200 OK	Lista JSON
GET detalle existente	200 OK	Detalle JSON
POST válido	201 Created	Registro creado
POST con peso 0	400 Bad Request	Error de validación
PATCH detalle	200 OK	Registro actualizado
DELETE detalle	204 No Content	Eliminación sin cuerpo
GET id inexistente	404 Not Found	Recurso no encontrado

La API devuelve datos para procesar desde otro programa; un Template devuelve HTML para mostrar una página. Django Admin sirve para la gestión interna. La práctica mantiene acceso abierto porque todavía no se configuraron permisos; antes de publicar la API se requiere autenticación y autorización.

Ejercicio 7 Recurso de solo lectura con relaciones anidadas

GitHub main / Semana_08/gestion_clinica/api_views.py | extracto de código

```
    serializer_class = ServicioSerializer


class CitaViewSet(viewsets.ModelViewSet):
    queryset = Cita.objects.select_related("paciente").all()
    serializer_class = CitaSerializer


class PacienteServicioViewSet(viewsets.ReadOnlyModelViewSet):
    """Atenciones de solo lectura, con relaciones precargadas para el JSON."""

    serializer_class = PacienteServicioSerializer

    def get_queryset(self):
        queryset = PacienteServicio.objects.select_related(
            "paciente",
            "paciente__propietario",
            "servicio",
        ).all()
        estado = self.request.query_params.get("estado")
        if estado:
            queryset = queryset.filter(estado__iexact=estado.strip())
```

PacienteServicioSerializer anida Paciente, Propietario y Servicio. PacienteServicioViewSet es de solo lectura, filtra por ?estado= y usa select_related("paciente", "paciente__propietario", "servicio") para recuperar las relaciones en una consulta.

La prueba test_select_related_reduces_nested_serialization_queries contiene aserciones que esperan la siguiente comparación para tres atenciones de ejemplo. No son resultados ejecutados en esta sesión:

Medición definida en la prueba	Consultas esperadas	Explicación
Sin optimizar	10	Una consulta de lista más tres consultas relacionadas por cada una de 3 atenciones.
Con select_related()	1	Una consulta SQL con JOIN para cargar paciente, propietario y servicio.

El comando python manage.py medir_api_queries imprime la medición real al ejecutarlo en el proyecto configurado; el total depende de los registros existentes.

Ejercicio 8 Búsqueda ordenamiento filtro y acción

GitHub main / Semana_08/gestion_clinica/api_views.py | extracto de código

```
class PacienteViewSet(viewsets.ModelViewSet):
    queryset = Paciente.objects.select_related("propietario").all()
    serializer_class = PacienteSerializer
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ["nombre", "especie", "raza", "propietario__nombres", "propietario__apellidos"]
    ordering_fields = ["nombre", "especie", "edad", "peso"]
    ordering = ["nombre"]

    @action(detail=False, methods=["get"], url_path="perros-pesados")
    def perros_pesados(self, request):
        pacientes = self.get_queryset().solo_perros().pesados()
…
    serializer_class = ServicioSerializer


class CitaViewSet(viewsets.ModelViewSet):
    queryset = Cita.objects.select_related("paciente").all()
    serializer_class = CitaSerializer


class PacienteServicioViewSet(viewsets.ReadOnlyModelViewSet):
    """Atenciones de solo lectura, con relaciones precargadas para el JSON."""

    serializer_class = PacienteServicioSerializer

    def get_queryset(self):
        queryset = PacienteServicio.objects.select_related(
            "paciente",
            "paciente__propietario",
            "servicio",
        ).all()
        estado = self.request.query_params.get("estado")
        if estado:
            queryset = queryset.filter(estado__iexact=estado.strip())
```

PacienteViewSet habilita SearchFilter y OrderingFilter. PacienteServicioViewSet filtra por estado. La acción perros-pesados reutiliza solo_perros() y pesados() del QuerySet de la Semana 7.

URL	Qué muestra
/api/pacientes/?search=Max	Coincidencias en nombre, especie, raza o propietario.
/api/pacientes/?ordering=-peso	Pacientes ordenados por peso descendente.
/api/atenciones/?estado=PAGADO	Atenciones pagadas.
/api/pacientes/?search=Max&ordering=nombre	Búsqueda y orden combinados.
/api/pacientes/perros-pesados/	Perros con peso de 15 kg o más.

Parte 2 API sobre la investigación propia

Ejercicio 9 Identificar recursos

GitHub main / Semana_08/gestion_clinica/api_views.py | extracto de código

```
    search_fields = ["nombre", "especie", "raza", "propietario__nombres", "propietario__apellidos"]
    ordering_fields = ["nombre", "especie", "edad", "peso"]
    ordering = ["nombre"]

    @action(detail=False, methods=["get"], url_path="perros-pesados")
    def perros_pesados(self, request):
        pacientes = self.get_queryset().solo_perros().pesados()
        serializer = self.get_serializer(pacientes, many=True)
        return Response(serializer.data)


class PropietarioViewSet(viewsets.ModelViewSet):
    queryset = Propietario.objects.all().order_by("apellidos", "nombres")
```

Apliqué la API a la Clínica Veterinaria del repositorio. El recurso simple es Paciente; el relacionado de solo lectura es PacienteServicio; el campo de estado es estado; y la acción reutiliza Paciente.objects.solo_perros().pesados(). Los modelos existentes se mantienen.

Ejercicio 10 Serializers y validación

GitHub main / Semana_08/gestion_clinica/serializers.py | extracto de código

```
class PacienteSerializer(serializers.ModelSerializer):
    """Serializer del recurso simple con validación de regla de negocio."""

    class Meta:
        model = Paciente
        fields = ["id", "propietario", "nombre", "especie", "raza", "edad", "peso"]

    def validate_peso(self, value):
        if value <= 0:
            raise serializers.ValidationError("El peso debe ser mayor que cero.")
```

Implementé ModelSerializers para Paciente, Propietario, Medicamento, Veterinario, HistorialMedico, Servicio, PacienteServicio y Cita. PacienteSerializer tiene validate_peso(). Un peso positivo se acepta; cero se rechaza con is_valid() igual a False y un error en errors["peso"].

Ejercicio 11 ViewSets router y estados

GitHub main / Semana_08/gestion_clinica/api_urls.py | extracto de código

```
    ServicioViewSet, VeterinarioViewSet,
)

router = DefaultRouter()
router.register("pacientes", PacienteViewSet, basename="paciente")
router.register("propietarios", PropietarioViewSet, basename="propietario")
router.register("medicamentos", MedicamentoViewSet, basename="medicamento")
router.register("veterinarios", VeterinarioViewSet, basename="veterinario")
router.register("historiales", HistorialMedicoViewSet, basename="historial")
router.register("servicios", ServicioViewSet, basename="servicio")
router.register("citas", CitaViewSet, basename="cita")
router.register("atenciones", PacienteServicioViewSet, basename="atencion")

urlpatterns = [
```

DefaultRouter publica los recursos bajo /api/. Paciente, Propietario, Medicamento, Veterinario, HistorialMedico, Servicio y Cita usan ModelViewSet. Las atenciones usan ReadOnlyModelViewSet. Las pruebas cubren 200, 201, 400, 404 y 204; un POST a atenciones se rechaza con 405.

Ejercicio 12 Relaciones anidadas y consultas

GitHub main / Semana_08/gestion_clinica/tests.py | extracto de código

```
            self.assertEqual(heavy_dogs.status_code, 200)
            self.assertEqual(len(heavy_dogs.data), 1)

            paid = self.client.get("/api/atenciones/?estado=PAGADO")
            pending = self.client.get("/api/atenciones/?estado=PENDIENTE")
            self.assertEqual(paid.status_code, 200)
            self.assertEqual(len(paid.data), 1)
            self.assertEqual(len(pending.data), 0)

        def test_nested_read_only_endpoint(self):
            response = self.client.get("/api/atenciones/")
            self.assertEqual(response.status_code, 200)
            self.assertEqual(response.data[0]["paciente"]["propietario"]["nombres"], "Ana")
            created = self.client.post(
                "/api/atenciones/",
                {"paciente": self.paciente.pk, "servicio": self.servicio.pk},
                format="json",
            )
            self.assertEqual(created.status_code, 405)


    class APIQueryOptimizationTests(TestCase):
        @classmethod
        def setUpTestData(cls):
            owner = Propietario.objects.create(
                nombres="Luis", apellidos="Paredes", dni="87654321",
                telefono="999222333", direccion="Arequipa",
            )
            servicio = Servicio.objects.create(
                nombre="Vacunacion", duracion_minutos=20, precio="30.00",
                cupos_disponibles=20,
            )
            for index in range(3):
                paciente = Paciente.objects.create(
                    propietario=owner, nombre=f"Perro {index}", especie="Perro",
                    raza="Mestizo", edad=index + 1, peso="16.00",
```

El endpoint /api/atenciones/ entrega Paciente, Propietario y Servicio dentro de cada atención. La prueba automatizada compara 10 consultas sin optimización con 1 consulta usando select_related(); esos valores son las aserciones del código y requieren ejecutar la suite para confirmarse.

Ejercicio 13 Filtros y acción

GitHub main / Semana_08/gestion_clinica/api_views.py | extracto de código

```
    class PacienteViewSet(viewsets.ModelViewSet):
        queryset = Paciente.objects.select_related("propietario").all()
        serializer_class = PacienteSerializer
        filter_backends = [filters.SearchFilter, filters.OrderingFilter]
        search_fields = ["nombre", "especie", "raza", "propietario__nombres", "propietario__apellidos"]
        ordering_fields = ["nombre", "especie", "edad", "peso"]
        ordering = ["nombre"]

        @action(detail=False, methods=["get"], url_path="perros-pesados")
        def perros_pesados(self, request):
            pacientes = self.get_queryset().solo_perros().pesados()
    …
        serializer_class = ServicioSerializer


    class CitaViewSet(viewsets.ModelViewSet):
        queryset = Cita.objects.select_related("paciente").all()
        serializer_class = CitaSerializer


    class PacienteServicioViewSet(viewsets.ReadOnlyModelViewSet):
        """Atenciones de solo lectura, con relaciones precargadas para el JSON."""

        serializer_class = PacienteServicioSerializer

        def get_queryset(self):
            queryset = PacienteServicio.objects.select_related(
                "paciente",
                "paciente__propietario",
                "servicio",
            ).all()
            estado = self.request.query_params.get("estado")
            if estado:
                queryset = queryset.filter(estado__iexact=estado.strip())
```

Se pueden buscar pacientes con ?search=, ordenarlos con ?ordering= y filtrar atenciones con ?estado=. Las URLs /api/atenciones/?estado=PAGADO y /api/pacientes/perros-pesados/ muestran el filtro y la acción. También se puede combinar ?search=Max&ordering=nombre.

Ejercicio 14 Publicar en GitHub

GitHub main / Semana_08/gestion_clinica/api_urls.py | extracto de código

```
    ServicioViewSet, VeterinarioViewSet,
)

router = DefaultRouter()
router.register("pacientes", PacienteViewSet, basename="paciente")
router.register("propietarios", PropietarioViewSet, basename="propietario")
router.register("medicamentos", MedicamentoViewSet, basename="medicamento")
router.register("veterinarios", VeterinarioViewSet, basename="veterinario")
router.register("historiales", HistorialMedicoViewSet, basename="historial")
router.register("servicios", ServicioViewSet, basename="servicio")
router.register("citas", CitaViewSet, basename="cita")
router.register("atenciones", PacienteServicioViewSet, basename="atencion")

urlpatterns = [
```

Creé Semana_08 usando Semana_07 como plantilla, agregué los archivos de la API, actualicé requirements.txt y añadí Semana 08 al README general sin quitar las secciones anteriores. El commit quedó publicado en main.

```
https://github.com/JesusCastillo2026/Desarrollo-de-Aplicaciones-Empresariales
Commit: b5b8dc1af536979ee8c0e35a0300045401874c8c
```

Conclusiones

En este laboratorio integré una API REST a la clínica reutilizando los modelos, relaciones y consultas ya desarrollados. Aprendí a convertir modelos a JSON, validar datos, exponer operaciones CRUD con ViewSets y registrar rutas con DefaultRouter. También incorporé filtros y optimicé la serialización de relaciones con select_related(). Antes de publicar la API en un entorno real debo configurar autenticación y permisos.

Evidencias de implementación

Las imágenes son extractos fieles del código versionado en Semana_08, con el nombre de archivo y los números de línea para localizarlo. Las imágenes de pruebas muestran las aserciones escritas; no son capturas de una ejecución. En este entorno no se ejecutó la suite porque faltan Django y Django REST Framework. Para comprobarlas en el proyecto, ejecute python manage.py test gestion_clinica.